Formative Assignment: Advanced Linear Algebra (PCA)

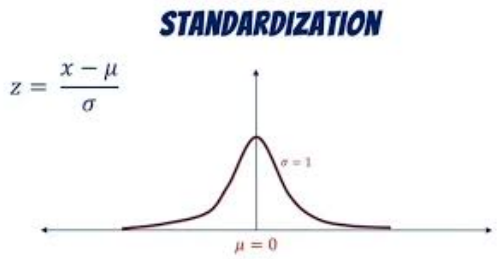
This notebook will guide you through the implementation of Principal Component Analysis (PCA). Fill in the missing code and provide the required answers in the appropriate sections. You will work with a dataset that is Africanized .

1. Make sure to display outputs for each code cell when submitting.
2. Do not write all your code on one cell
3. Do not use any libraries aside from numpy

Step 1: Load and Standardize the Data

Before applying PCA, we must standardize the dataset. Standardization ensures that all features have a mean of 0 and a standard deviation of 1, which is essential for PCA. Fill in the code to standardize the dataset.

STRICTLY - Write code that implements standardization based on the image below




```
import numpy as np
import pandas as pd

# Load dataset
data = pd.read_csv("Country-data.csv")

# Remove non-numeric column
numeric_data = data.drop("country", axis=1)

# Convert to numpy array
X = numeric_data.values

# Standardization using NumPy only
mean = np.mean(X, axis=0)
std = np.std(X, axis=0)

standardized_data = (X - mean) / std

standardized_data[:5]
```

```
array([[ 1.29153238, -1.13827979,  0.27908825, -0.08245496, -0.8082454 ,
         0.15733622, -1.61909203,  1.90288227, -0.67917961],
       [-0.5389489 , -0.47965843, -0.09701618,  0.07083669, -0.3753689 ,
        -0.31234747,  0.64786643, -0.85997281, -0.48562324],
       [-0.27283273, -0.09912164, -0.96607302, -0.64176233, -0.22084447,
         0.78927429,  0.67042323, -0.0384044 , -0.46537561],
       [ 2.00780766,  0.77538117, -1.44807093, -0.16531531, -0.58504345,
         1.38705353, -1.17923442,  2.12815103, -0.51626829],
       [-0.69563412,  0.1606679 , -0.28689415,  0.4975675 ,  0.10173177,
        -0.60174853,  0.70425843, -0.54194633, -0.04181713]])
```

Step 3: Calculate the Covariance Matrix

The covariance matrix helps us understand how the features are related to each other. It is a key component in PCA.

```
cov_matrix = np.cov(standardized_data, rowvar=False)

cov_matrix
```

```
array([[ 1.0060241 , -0.32000945, -0.2016093 , -0.12797725, -0.52747354,
         0.29001277, -0.89201752,  0.85358942, -0.485942  ],
       [-0.32000945,  1.0060241 , -0.11509761,  0.74182289,  0.51989676,
        -0.10794074,  0.3182181 , -0.32193832,  0.42124719],
       [-0.2016093 , -0.11509761,  1.0060241 ,  0.09629328,  0.1303592 ,
        -0.2569142 ,  0.21196135, -0.19785877,  0.34804965],
       [-0.12797725,  0.74182289,  0.09629328,  1.0060241 ,  0.12314364,
        -0.2484822 ,  0.05471819, -0.16000656,  0.11619394],
       [-0.52747354,  0.51989676,  0.1303592 ,  0.12314364,  1.0060241 ,
        -0.14864609,  0.61564899, -0.50486319,  0.90096644],
       [ 0.29001277, -0.10794074, -0.2569142 , -0.2484822 , -0.14864609,
         1.0060241 , -0.24114897,  0.31883023, -0.22296618],
       [-0.89201752,  0.3182181 ,  0.21196135,  0.05471819,  0.61564899,
        -0.24114897,  1.0060241 , -0.76545827,  0.60370413],
       [ 0.85358942, -0.32193832, -0.19785877, -0.16000656, -0.50486319,
         0.31883023, -0.76545827,  1.0060241 , -0.45765069],
       [-0.485942  ,  0.42124719,  0.34804965,  0.11619394,  0.90096644,
        -0.22296618,  0.60370413, -0.45765069,  1.0060241 ]])
```

The importance of the covariance matrix lies in its ability to help discover correlations within variables and indicate the degree to which they vary together. The covariance matrix enables one to establish directions along which there is the highest variation in the data set. Moreover, the covariance matrix is helpful in minimizing redundancy among the highly correlated variables.

Step 4: Perform Eigendecomposition

Eigendecomposition of the covariance matrix will give us the eigenvalues and eigenvectors, which are essential for PCA. Fill in the code to compute the eigenvalues and eigenvectors of the covariance matrix.

```
# Step 4: Perform Eigendecomposition
eigenvalues, eigenvectors = None  # Perform eigendecomposition
eigenvalues, eigenvectors
```

Step 5: Sort Principal Components

Sort the eigenvectors based on their corresponding eigenvalues in descending order. The higher the eigenvalue, the more important the eigenvector. Complete the code to sort the eigenvectors and print the sorted components.

How Is Explained Variance Used In PCA?'

```
sorted_indices = np.argsort(eigenvalues)[::-1]

sorted_eigenvalues = eigenvalues[sorted_indices]

sorted_eigenvectors = eigenvectors[:, sorted_indices]
```

```
sorted_eigenvectors
```

```
array([[ 0.41951945, -0.19288394,  0.02954353, -0.37065326, -0.16896968,
         -0.20062815, -0.07948854, -0.68274306, -0.3275418 ],
        [-0.28389698, -0.61316349, -0.14476069, -0.00309102,  0.05761584,
          0.05933283, -0.70730269, -0.01419742,  0.12308207],
        [-0.15083782,  0.24308678,  0.59663237, -0.4618975 ,  0.51800037,
         -0.00727646, -0.24983051,  0.07249683, -0.11308797],
        [-0.16148244, -0.67182064,  0.29992674,  0.07190746,  0.25537642,
          0.03003154,  0.59218953, -0.02894642, -0.09903717],
        [-0.39844111, -0.02253553, -0.3015475 , -0.39215904, -0.2471496 ,
         -0.16034699,  0.09556237,  0.35262369, -0.61298247],
        [ 0.19317293,  0.00840447, -0.64251951, -0.15044176,  0.7148691 ,
         -0.06628537,  0.10463252, -0.01153775,  0.02523614],
        [-0.42583938,  0.22270674, -0.11391854,  0.20379723,  0.1082198 ,
          0.60112652,  0.01848639, -0.50466425, -0.29403981],
        [ 0.40372896, -0.15523311, -0.01954925, -0.37830365, -0.13526221,
          0.75068875,  0.02882643,  0.29335267,  0.02633585],
        [-0.39264482,  0.0460224 , -0.12297749, -0.53199457, -0.18016662,
         -0.01677876,  0.24299776, -0.24969636,  0.62564572]])
```

Step 6: Project Data onto Principal Components

Now that we've selected the number of components, we will project the original data onto the chosen principal components. Fill in the code to perform the projection.

```
explained_variance_ratio = sorted_eigenvalues / np.sum(sorted_eigenvalues)

cumulative_variance = np.cumsum(explained_variance_ratio)

num_components = np.argmax(cumulative_variance >= 0.90) + 1

principal_components = sorted_eigenvectors[:, :num_components]

reduced_data = np.dot(standardized_data, principal_components)

reduced_data[:5]
```

```
array([[ 2.91302459, -0.09562058,  0.7181185 , -1.00525464, -0.15831004],
       [-0.42991133,  0.58815567,  0.3334855 ,  1.16105859,  0.17467732],
       [ 0.28522508,  0.45517441, -1.22150481,  0.8681145 ,  0.15647465],
       [ 2.93242265, -1.69555507, -1.52504374, -0.83962501, -0.27320893],
       [-1.03357587, -0.13665871,  0.22572092,  0.84706269, -0.19300696]])
```

Step 7: Output the Reduced Data

Finally, display the reduced data obtained by projecting the original dataset onto the selected principal components.

```
print(f"Reduced Data Shape: {reduced_data.shape}")

reduced_data[:5]
```

```
Reduced Data Shape: (167, 5)
array([[ 2.91302459, -0.09562058,  0.7181185 , -1.00525464, -0.15831004],
       [-0.42991133,  0.58815567,  0.3334855 ,  1.16105859,  0.17467732],
       [ 0.28522508,  0.45517441, -1.22150481,  0.8681145 ,  0.15647465],
       [ 2.93242265, -1.69555507, -1.52504374, -0.83962501, -0.27320893],
       [-1.03357587, -0.13665871,  0.22572092,  0.84706269, -0.19300696]])
```

Step 8: Visualize Before and After PCA

Now, let's plot the original data and the data after PCA to compare the reduction in dimensions visually.

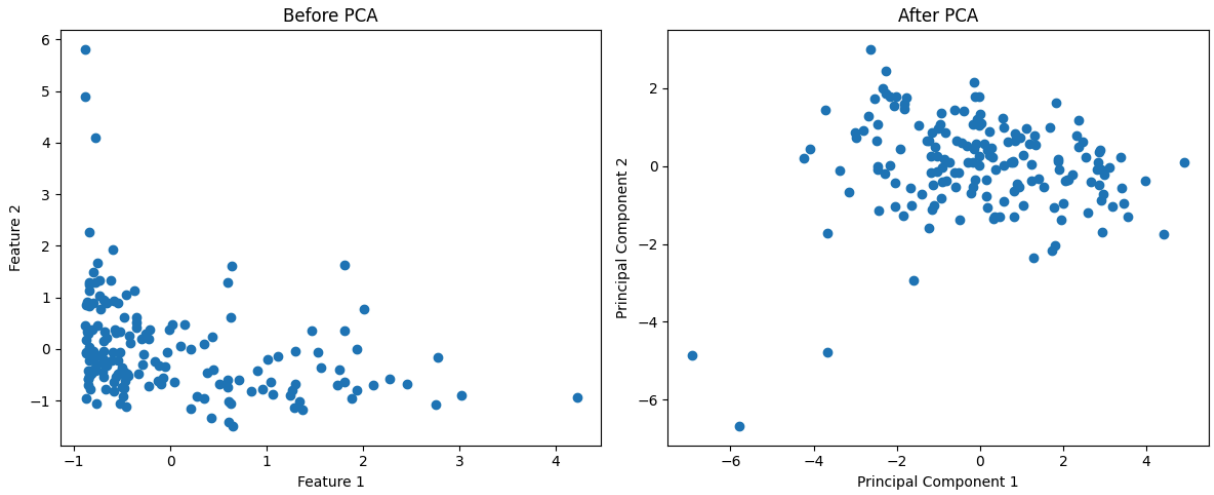
```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))

# Original Data
plt.subplot(1,2,1)
plt.scatter(
    standardized_data[:,0],
    standardized_data[:,1]
)
plt.title("Before PCA")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")

# Reduced Data
plt.subplot(1,2,2)
plt.scatter(
    reduced_data[:,0],
    reduced_data[:,1]
)
plt.title("After PCA")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")

plt.tight_layout()
plt.show()
```



Please make sure you do not use more than 5 lines to answer each of the following points:

1. Interpret the Visual you just created of the before and after PCA

> The "Before PCA" graph is a graphical representation of the data with the help of the initial variables whose relationship might not be easy to find out due to their multidimensional nature. In contrast, the "After PCA" graph presents data through the use of principal components which contain maximum variation of the data.

2. Explain why you selected the number of principle components, more especially explain what tradeoffs you are making.

> Five Principal Components have been chosen since they represent around 94.5% of the total variance in the data set. It implies that the majority of information in the original 9-dimensional space has been retained after dimensionality reduction to a 5-dimensional space. The price to be paid for this, on the other hand, is the loss of a certain amount of information.

3. Clearly mention based on your use case/ dataset ("economic activity", "population pressure"), What information is lost when reducing dimensions?

> In the case of the given dataset on economics activities, there is aggregation of many variables into fewer principal components by PCA. This leads to the blurring of specific features of particular variables, like inflation rate, export rate, or fertility rate. Small differences between the countries might also be overlooked due to the process used. It means that some features of particular economies get lost in return for simplicity.